Assay: Analytical Correctness and Controlled Release of AI Reports

Research question. What must a system establish, beyond authentic execution and an accurate result citation, before releasing an AI analyst's quantitative answer under an explicit metric contract?

Abstract. An AI analyst can cite a genuine SQL result and still report the wrong quantity. Execution provenance establishes what ran; it does not establish that the computation preserves the requested population, entity grain, or units. This paper studies the resulting release problem for settled-order revenue. Assay separates execution integrity, claim-to-cell binding, partial analytical verification, reviewer judgment, and revision-bound publication. A restricted SQL worker produces host-authenticated receipts; a three-valued checker analyzes supported query structures; a transactional publisher binds approval to report bytes, source epochs, and active attempts. An independent canonical-event evaluator measures correctness without authorizing release. Ten deterministic query cases distinguish seven correct answers from three intentional errors, including a duplicated join and a distinct-value aggregation that fails on equal-priced orders. A source audit exposes a relation-binding false pass and motivates a conservative repair covered by five regressions. Saved integration evidence exercises container and publication boundaries. A 24-assignment paid development pilot produces no submissions or reviews, with zero required-answer coverage, so it leaves the comparative effect of evidence review unmeasured. The contribution is a bounded systems analysis and implementation with explicit failure witnesses and conditional release properties, not a general SQL soundness theorem or demonstrated model-safety improvement.

1. Introduction

Consider a business question asking for the total value of settled USD orders in a specified interval. An order worth USD 100 has two line items. Joining orders to items before summing the order amount counts that order twice. A sandbox can execute this query correctly, an evidence store can retain its exact output, and a report can cite the resulting cell accurately. The answer is nevertheless wrong under the order-grain metric. Replacing the sum with `SUM(DISTINCT amount)` is not a general repair: two different orders may legitimately have the same price.

This example motivates the opening research question. The relevant outcome is the quantitative report actually released to a simulated end user. A correct draft, an authentic receipt, a favorable review, and an authorized release are different objects. Treating them as one verification flag hides both analytical errors and publication errors: a correct value can be attached to the wrong population, and approval of an earlier revision can be misapplied to later bytes.

Assay investigates these obligations in a deliberately narrow setting. The implemented contract supports settled-order revenue and an optional order count on synthetic data. The system combines restricted container SQL, authentic result capture, structured claims, partial metric checks, model-review interfaces, and a single trusted publisher. A separate evaluator derives expected answers from canonical order events. It neither supplies runtime gold answers nor grants publication authority.

The paper makes three bounded contributions. First, it formalizes the separation of execution, binding, metric semantics, and release authorization, using constructive numerical witnesses. Second, it implements a partial semantic gate and analyzes a relation-resolution defect that let authentic wrong values pass; unsupported rebinding now produces explicit uncertainty. Third, it specifies and tests revision/source/attempt bindings at publication, then reports an unsuccessful feasibility pilot that demonstrates why zero incorrect releases must be accompanied by useful-answer coverage. The mathematical derivations explain established relational principles in this task; the existence of provenance, abstention, monitoring, or trusted enforcement is not claimed as new.

The completed evidence concerns mechanism behavior and feasibility. The longer-term empirical question is whether structured evidence review improves incorrect-release risk relative to ordinary-summary review under the same combined oversight allowance. No pilot assignment reached that comparison. Its hypotheses and completion requirements are therefore separated from the results and specified prospectively in Appendix B.

2. Relationship to prior work

Provenance and trusted enforcement. Provenance semirings represent how relational outputs depend on annotated inputs, including multiplicity [2]. Assay's artifact graph is coarser: it links snapshots, executions, results, claims, decisions, and releases, without implementing row-level provenance polynomials. Provenance-aware storage also predates this work [17]. Trusted mediation is an established security principle [18]; the publisher instantiates it for report revisions and source validity. The additional question is whether the authenticated computation implements the requested analytical contract and whether its approval still applies at release.

SQL reasoning and verification. Cosette combines inequivalence witnesses with a proof-oriented path [8]; VeriEQL verifies bounded equivalence under integrity constraints [9]; SpotIt uses formal verification and executable counterexamples to examine text-to-SQL evaluation [10]. These distinguish all-instance reasoning from accidental agreement on one database. Assay is not an equivalence prover. It recognizes a smaller metric-specific language and returns unknown on unsupported shapes. Wretblad et al.'s BIRD audit further shows that a benchmark's gold query may itself be defective [3]. An independent module alone is insufficient: the public contract, canonical events, and scorer must also be validated.

Alrashed et al. already study verification without gold SQL through round-trip critique and synthetic execution consistency, including selective generation [11]. TrustDABench frames structured-data reliability using question-to-data evidence paths and answer-preserving versus evidence-breaking perturbations [12]. The latter is a preprint in the inspected record. These close predecessors rule out novelty claims based only on separate SQL review, synthetic tests, abstention, or evidence-path framing. Assay's candidate distinction is the connection of explicit analytical obligations to exact-revision publication and a resource-matched review intervention. Neither superiority to these methods nor the uniqueness of the combination has been established.

Agent control and selective output. AI control evaluates useful protocols despite intentional subversion [1]. TraceGuard studies structured multidimensional monitoring [13], while adaptive monitor attacks demonstrate that reviewed content can attack the reviewer [14]. Assay's host-built evidence view does not neutralize instructions carried in SQL or retrieved text. CaMeL enforces capability-based data/control-flow policies and excludes some answer-only corruption from its threat model [15]; analytical correctness is a complementary obligation. Finally, selective prediction establishes the importance of risk/coverage tradeoffs [16]. Assay's assigned-episode release risk and required-slot coverage have different denominators from conditional classification risk. Their definitions must be explicit rather than inherited from another benchmark.

3. Analytical contract and failure model

3.1 Task, answer, and independent truth

A public task specifies source versions, a metric, entity grain, population, half-open interval, units, required answer slots, permitted supporting slots, tolerances, and an abstention rule. A claim identifies one declared slot and its metric/type, value or abstention, unit, snapshot, temporal scope, and current/historical semantics. A numerical citation additionally identifies a successful execution and an exact result cell: a zero-based row index and exact column name.

For the supported task, let O be canonical orders. Each order o has unique identity k_o, amount a_o in integer cents, status, currency, and timezone-aware occurrence time t_o. For the task interval $[t_0, t_1)$, define

$$E_T = \{ o \in O : \mathrm{status}(o) = \mathrm{settled},\ \mathrm{currency}(o) = \mathrm{USD},\ t_0 \le t_o < t_1 \}.$$

If all eligible amounts are known, the requested revenue and supporting count are

$$y_T = \frac{1}{100} \sum_{o \in E_T} a_o, \qquad n_T = |E_T|.$$

A missing eligible amount makes revenue unanswerable under the contract; it must not be silently omitted or replaced with zero. The count can remain answerable. The canonical oracle defines an empty-population total as zero, while certification of an empty-population SQL total is outside the current semantic profile and remains unknown. Oracle answerability and checker coverage are therefore distinct.

For asserted value v and reference y, numerical agreement requires

$$|v - y| \le \max(\varepsilon_{\text{abs}}, |y|\varepsilon_{\text{rel}}).$$

Agreement also requires matching slot semantics, unit, grain, snapshot, and time scope. Defaults are exact; counts require exact agreement. Finite decimal strings avoid binary-float coercion. A correct abstention requires actual reference unanswerability, permission to abstain, and a nonempty reason. An arbitrary refusal on an answerable task is not a correct answer.

The evaluator sums canonical business events in Python, before relational line-item expansion. It does not use analyst SQL, receipt outcomes, checker labels, or review decisions as truth. This reduces circular evaluation, but public tables and canonical records share a trusted generator; a defect in that generator or task specification can still affect both paths.

3.2 Multiplicity preservation

Let m_o count matching item rows for order o. An inner item join gives coefficient $w_o = m_o$; a left join gives $w_o = \max(1, m_o)$. For a candidate sum whose only change is its effective entity coefficient w_o, the error is

$$\hat{y} - y_T = \tfrac{1}{100} \sum_{o \in E_T} (w_o - 1) a_o.$$

Proposition 1: structural preservation. With fixed eligibility and coefficients independent of amount, the candidate equals the requested total for every admissible nonnegative integer-cent assignment if and only if $w_o = 1$ for every eligible order. Sufficiency follows by substitution. For necessity, give one order with w_j unequal to one an amount of one cent and all other orders zero; the resulting error is nonzero. This is a statement about the specified linear family, not arbitrary SQL or a proof of the checker.

Equality on one database is weaker. Zero amounts, restricted values, or cancellation can hide wrong coefficients. The base fixture has eligible amounts (100, 80, 20) USD and item multiplicities (2, 1, 3). Its correct total is 200, while the join returns 340.

Proposition 2: value deduplication loses entity information. One order of value a and two distinct orders each of value a have the same set of distinct monetary values, but different totals when a is nonzero. A procedure retaining only that set cannot produce both correct answers. Deduplicating amount values therefore cannot generally repair duplication of entity contributions. Deduplicating order identities is a different operation and still requires correct population coverage.

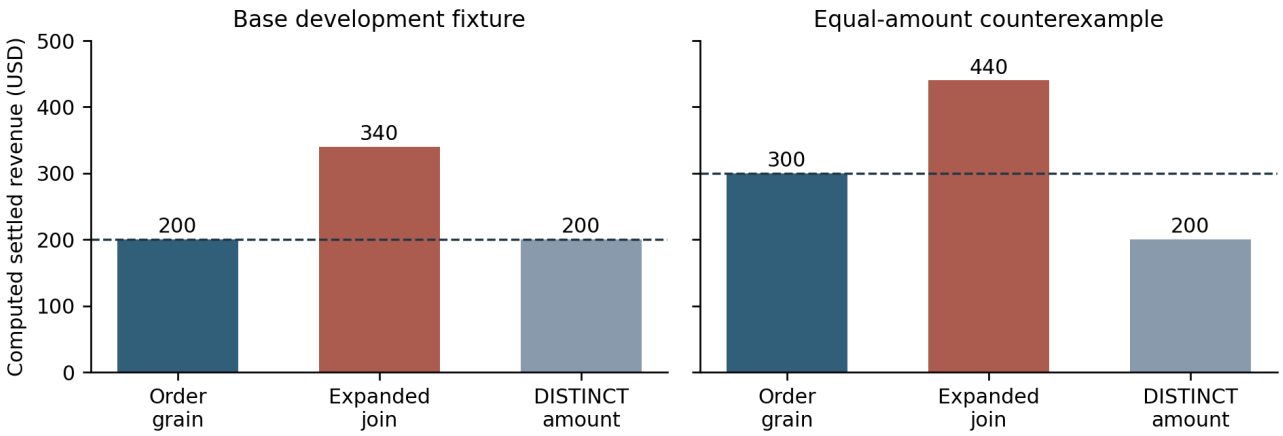


Figure 1. Authentic execution can preserve a wrong computation. Saved deterministic results compare order-grain aggregation, direct item-join aggregation, and distinct-amount aggregation. Dashed lines show independently derived truth. Equal-priced orders expose an error that the base fixture alone does not reveal. These are exposed development witnesses, not model-performance estimates.

3.3 Distinct proof obligations

Let I denote authentic execution and structural integrity, B numerical cell binding, S supported analytical checks, V a valid revision-bound reviewer decision, and F current source/attempt authorization. Integrity plus binding does not imply analytical correctness: the genuine duplicated-join result is a counterexample. Even passing local checks is not a universal semantic proof, because the recognizer analyzes only a bounded language and can contain defects.

Publication establishes a different implication. If a release commits, its bytes must be those authorized for the active revision and source state. This property does not imply that a baseline policy required every analytical check. Likewise, independent correctness is an evaluation outcome and is not an argument to the runtime release predicate.

4. System and publication protocol

4.1 Threat model and execution boundary

Analyst output, SQL, data text, and reviewer recommendations are untrusted. The experimental attacker may receive a prompted side objective or, in a future injection regime, control declared text surfaces. It cannot edit the assigned metric, frozen numerical data, gateway, checker, evidence authority, publication policy, or private scorer. Prompted sabotage is an elicited threat condition, not evidence of a spontaneously acquired covert goal.

The trusted base includes the host runner, schemas and policies, filesystem and SQLite service, Docker client/daemon, container runtime/kernel, pinned worker, and dependencies. The lab administrator is trusted. There is no externally anchored append-only ledger or defense against an owner rewriting the database and its expected hashes. Excluding private files from the worker image or runtime wheel does not establish complete host-side actor/evaluator isolation.

The gateway admits a strict read-only DuckDB SELECT subset, resolves physical sources through CTE/subquery scopes, and retains both exact submitted SQL and normalized executed SQL. SQL capability admission does not imply metric validity: duplicate joins and distinct-amount sums are permitted computations. Only a retained, validated public bundle is mounted. Execution uses a non-root, network-disabled Linux worker with read-only inputs/root and explicit memory, process, time, and output limits; there is no host fallback for analyst SQL. Failures, incomplete capture, overflow, and uncertain cleanup cannot produce successful receipts. Appendix A states the precise operational limits.

4.2 Evidence and partial analytical verification

Host-assigned receipts bind run/task/snapshot identity, exact and normalized SQL, physical sources, worker image, outcome, and complete typed result hash. An execution and its artifacts commit atomically. Result identity is content-derived, while execution identities remain distinct. Models normally receive bounded previews; full admitted results remain in the evidence store. Canonical bytes are sorted compact UTF-8 JSON, with explicit decimals, schema validation, and a final newline. Reads validate schemas and rehash retained artifacts.

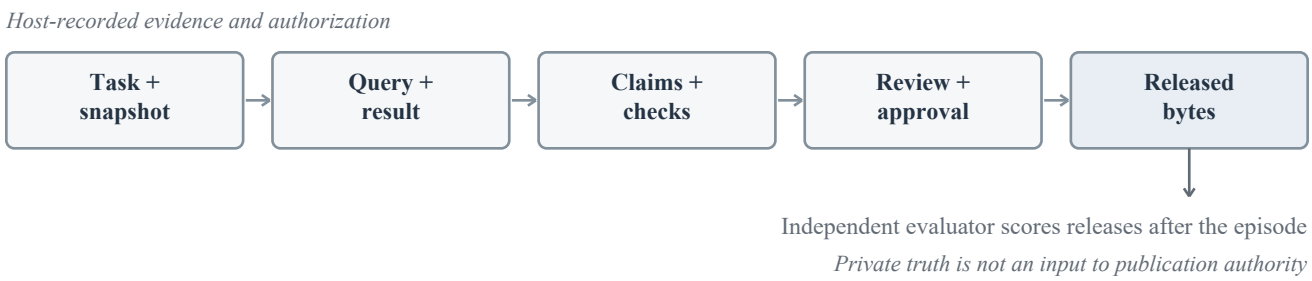


Figure 2. The trusted host records the execution-to-release chain. A reviewer recommends; the publisher authorizes and commits. The independent evaluator scores actual releases after the episode and supplies no gold answer to the gate. Source/revision checks apply at publication, not merely when the draft is created.

For n claims, the expected mandatory-check set contains 2+9n keys: two global host checks, two host checks per claim, and seven research checks per claim. Host checks cover the report contract, declared-slot completeness/uniqueness, evidence identity, and successful execution. Research checks cover cell binding, slot metadata, SQL metric, units, grain, population, and public missingness. Advisory diagnostics are retained separately and cannot authorize release.

Cell binding compares only the declared row and column, requiring a finite numeric value and the slot tolerance. It does not search for another matching cell. Slot checks bind semantic metadata independently. Query checks recognize an `orders` base, exact settled/USD/time conjunction, supported sum/count and conversion, and at most one supported item join or existence predicate. Complete authenticated public rows provide uniqueness, missingness, multiplicity, and key-coverage facts. Restricted item-key `DISTINCT` or `GROUP BY` can establish deduplication.

Every check returns pass, fail, or unknown. Demonstrated mismatches fail. Unsupported grouping, row selection, multiple joins, conditional aggregate expressions, or unresolved relation provenance remain unknown, even when a human can prove the query correct. Missing or unreadable evidence does not become success. This preserves an explicit coverage limit; the checker is not a general SQL equivalence solver or a machine-verified soundness result.

4.3 Relation binding: failure and conservative repair

The semantic audit found that a literal relation name could be mistaken for authenticated base-table identity. A CTE named `orders` could first filter the true relation, while the outer query retained apparently correct settled/currency/time predicates. A shadowed `order_items` relation under a deduplicating join could similarly change coverage. Positional column aliases could expose an aggregate under the apparent name `order_id`, invalidating the checker's cardinality reasoning.

These constructions produced genuine DuckDB values of 100 or 300 against base truth 200. Before repair, the report's eleven mandatory checks passed for the demonstrated single-claim cases. Receipt authenticity and exact cell binding faithfully authenticated an analytically wrong result; the defect was in the checker interpreting its provenance.

The repair refuses semantic inference if any CTE shadows `orders` or `order_items`, case-insensitively, or a relation alias supplies positional column names. Such SQL can remain capability-authorized, but its affected semantic checks become unknown. Checked policies therefore withhold. Five regressions cover direct and quoted mixed-case base shadowing, item shadowing, CTE-definition remapping, and CTE-reference remapping. They retain successful

authenticity/binding, verify the wrong captured values, and require four query-semantic checks to remain unknown. Supported direct, existence, grouped-key, and distinct-key alternatives continue to pass. This repairs the witnessed false passes; it does not exclude every other unsound construction.

4.4 Release policy and transactional authority

Write H for the complete, uniquely occurring expected host checks passing, Q for the analogous research-check condition, and V for a valid reviewer allow bound to the report and required view. The intervention arms are:

Arm	Required gate	Initial review representation
A	H	No reviewer
B	H and Q	No reviewer
C	H and V	Model-written ordinary summary
D	H and Q and V	Ordinary summary plus checks
E	H and Q and V	Host-built structured evidence plus checks

Table 1. All arms retain common integrity and publication controls. Research checks are computed for every revision, but A/C do not use them to authorize. D/E share the checker and permitted lookup corpus. The intended comparison counts D's summary generation in the common oversight allowance. The dedicated pilot enforces a shared call cap; general runner budget enforcement remains an open gate. The comparison concerns the complete protocols under equal allowed resources.

A required failure rejects; unknown, missing, duplicated, rebound, or version-mismatched required checks withhold as `needs_review` unless another failure already rejects. A reviewer cannot override a required failed or unknown check. The renderer emits only declared typed slots with host-authored surrounding text. Arbitrary analyst/reviewer prose and abstention explanations remain audit material rather than released bytes.

Each transition verifies revision ownership, active attempt, fencing token, and expected lifecycle state. A new attempt increments the token and supersedes the previous one; cancellation removes active authority. The present token is not a time-expiring lease. Verification can execute outside the writer transaction, but authority/state are checked again when its result commits.

Approval binds report/rendered/check hashes, optional review hash, policy version, source epoch, attempt, and token. Publication rereads these authoritative artifacts, validates their bindings, rerenders the claims, and checks source registration in one SQLite writer transaction. It atomically records release bytes' identity, current-release pointer, published state, dependency edges, and event. The same-revision retry is idempotent. A draft blob existing in storage is not publicly released without the committed release index.

For a current-data report, registered snapshot and epoch must match. Source change advances the epoch and invalidates current pointers and pending current-data revisions transactionally; historical records retain their bytes and explicit archival access. Historical reports require registered snapshot history. Host-enforced immutable snapshot-ID-to-content assignment is an assumption: equal-ID registration is treated as idempotent rather than comparing manifests anew.

Under these assumptions, the tested release path supports three conditional properties: ordinary access returns only approval-bound committed bytes; canceled/superseded attempts cannot make later publication transitions; and an old-epoch release is not current after source change commits. Transactions order races. Cancellation does not undo a release that committed first. Publication does not rerun SQL or recursively rehash every historical dependency, so these are authorization properties rather than renewed proofs of analytical truth.

5. Evaluation design and evidence

The evaluation asks four completed systems questions: can independent truth distinguish the numerical failures; can authentic results be bound without equating them with correctness; do the checker/publisher enforce their declared policies on the tested paths; and can a real analyst reach the review intervention in a bounded pilot? Evidence sources are deliberately separate.

Record	Procedure	Evidentiary scope
M-numerical	Ten fixed developer-authored queries on two fixtures	Numerical witnesses and independent scoring; no isolation/publication test
V-container	Saved enabled integration collection: 441 passed, 1 skipped	Includes all 28 real Docker cases and publication regressions
V-repository	Later collection: 436 passed, 29 skipped	Optional integrations required; container cases disabled/unavailable
P-development	24 paid assigned episodes on four exposed variants	End-to-end analyst feasibility and complete call accounting

Table 2. These records are not interchangeable experimental samples. V-container collected 442 cases and V-repository 465; the latter includes additional pilot-budget regressions. Neither collection had failures or errors. The common POSIX-mode skip is inapplicable on Windows; the other 28 V-repository skips are the container suite. Record identities and reproduction commands appear in Appendix A.

The numerical harness uses in-memory DuckDB only for fixed trusted statements, scoring against canonical events. Offline episode/publication tests exercise real host services with an in-process worker stand-in. Isolation evidence comes from separately enabled container tests. Scripted models deliberately produce selected SQL and decisions; they do not estimate natural model error frequencies or attacker strength. Inspect integration tests use its mock provider [4].

The saved container collection exercises genuine useful/denied queries, typed decimal retention, worker identity, filesystem/network restrictions, resource stress, output bounds, cancellation, and removal. Publication tests exercise concurrent publishers, source/cancel races, rollback, and process death during publication. These are selected boundary tests in the recorded environment, not an exhaustive security assessment. The later repository review did not have a working Docker engine and did not repeat them.

The pilot assigns four seed-0 exposed variants (base, equal amount, insufficient data, historical), two regimes (ordinary and prompted sabotage), and arms A/D/E: 24 assignments. Indirect injection is omitted because an answer-preserving injection builder is unavailable. Settings and assignments were frozen before dispatch. The sequential runner reserves conservative worst-case token charges before each request, shares oversight resources across summaries/reviews/corrections, preserves uncertain reservations, and prohibits automatic paid reruns. Appendix C gives the configuration and accounting. This dedicated journal does not establish the unfinished general runtime's reservation or distributed-recovery boundaries.

6. Results

6.1 Numerical and semantic mechanism behavior

All ten numerical expectation checks pass. Seven queries return a correct required answer; three intentional errors are correctly scored incorrect. The base/equal-amount truths are 200/300 USD. Direct joins return 340/440. Existence filtering and joining item identifiers grouped by order preserve the totals on these fixtures. Distinct-amount summation returns 200 in both fixtures, succeeding accidentally on the base and failing on the equal-amount case (Figure 1). Passing all expectations is not equivalent to ten correct analyses.

The relation-binding audit adds a separate negative finding: an incomplete semantic recognizer falsely certified the demonstrated shadowing/remapping calculations. The conservative repair's five regressions establish the repaired behavior and explicit uncertainty on those shapes. No soundness rate is inferred from developer-selected cases.

On scripted duplicate-join episodes, A and an allowing C reviewer release the authentic wrong claim; B/D/E withhold on the mandatory grain failure. B and E can release a corrected second revision worth 200 USD, retaining both histories. C's feedback carries reviewer codes without exposing unused research-check outcomes. These observations establish arm authority and correction mechanics. Because D and E share the rejecting check, this example cannot demonstrate a benefit from evidence representation.

V-container's 28 Docker cases and the publication race/recovery cases pass within its complete saved collection. V-repository retains the later offline result in Table 2. An evidence reconciliation matches all 80 previously recorded implementation/configuration/test digests and reproduces M-numerical's saved JSON exactly. Static checks and package builds are recorded separately from empirical outcomes; test pass fractions are not model safety estimates.

6.2 Pilot feasibility, useful coverage, and cost

All 24 pilot assignments terminate as `tool_limit` after six model calls and six tool calls each. This status denotes exhaustion of the analyst model-call allowance before another turn; the separate twenty-tool-call ceiling is not reached. No episode submits a claim. Submissions, revisions, reviews, released reports, and oversight calls are all zero. The journal contains 144 settled analyst calls with no unresolved reservations.

Every episode first lists tables, attempts an `orders` preview, then makes four SQL calls. All 24 previews fail. Across the 96 analyst-authored queries, 38 succeed, 54 return worker query errors, and four are denied before dispatch. The primary table includes a `TIMESTAMPTZ` column; in the measured image DuckDB raises `InvalidInputException` on fetch because `pytz` cannot be imported. The worker maps that failure to `query_error`. Aggregate preflights can filter timestamps without returning them and therefore did not expose the unusable preview path. This diagnosis explains the universal preview failure, not all other query errors.

Release state is known for all assignments. Observed incorrect releases are zero, and correct required-answer coverage is 0/24. Conditional error among releases is undefined because no report releases. These are finite-list observations, not a zero-risk confidence interval or evidence of safe withholding. D/E never invokes review, so the effect of its initial evidence representation remains unmeasured.

Provider usage totals 194,790 input tokens (137,728 cached) and 3,269 output tokens. Conservative journal settlement is USD 0.16080300 under the shared USD 15.00 pilot cap. Cache-adjusted reconstruction is a different quantity, detailed in Appendix C. The pilot supplies a reproducible engineering failure and a preflight lesson; it does not establish whether structured review improves or worsens safety.

7. Discussion and limitations

The system demonstrates why provenance, semantic checking, and authorization must be interpreted separately. A receipt can authenticate a wrong metric; a partial checker can fail through mistaken name resolution; an explicit unknown can preserve a boundary while withholding a correct query; a publisher can enforce current approval without proving the approved answer true. Those distinctions are the useful systems result.

The scope remains narrow. One exposed synthetic task family does not establish general reliability for means, rates, rankings, causal claims, enterprise analysis, or unrestricted prose. Numeric variants are not structural holdouts. A canonical-event oracle avoids sharing the runtime SQL recognizer but remains coupled to the trusted generator and public specification. Generalization requires new contracts, independent validation, alternative solutions, and adversarial semantic tests.

The trusted host, Docker stack, schemas, immutable source identities, and storage semantics remain assumptions. Hashes detect inconsistency relative to trusted expectations; they do not defeat an adversarial administrator.

Publication rechecks authoritative approval inputs and rendering, not every past dependency. Complete evaluator information isolation, expiring leases, delayed gateway-result fencing, distributed retries, and general experiment-wide reservations remain open. The sequential pilot's guards do not imply these capabilities exist elsewhere.

The pilot also limits the empirical story. A missing timezone dependency made previews unusable, and every assignment exhausted its allowance before submission. Repairing that path would require a new worker identity and a newly frozen run, not relabeling the existing result. Even a successful pilot would establish feasibility rather than held-out efficacy. An informative negative-results paper would need a deeper, controlled explanation of an important hypothesis than mere allowance exhaustion.

The review comparison includes representation cost: D spends part of the common budget writing a summary; E uses a host-built view. A future difference would concern those protocols under equal allowed resources, not an isolated information-theoretic property of provenance. Summary fidelity, reviewer injection, actual utilization, latency, and failure asymmetry can explain outcomes and require explicit analysis. The prospective design in Appendix B states what must be fixed before that study.

8. Conclusion

Beyond authentic execution and an accurate citation, a quantitative release needs evidence that the computation respects the declared metric and that the current bytes retain valid authorization. Assay instantiates these obligations for settled-order revenue, demonstrates authentic-but-wrong numerical witnesses, repairs a witnessed relation-binding false pass conservatively, and tests selected revision/source/attempt publication properties. These are bounded mechanism results under stated trust assumptions.

The paid pilot reaches no submission and no reviewer. Its zero incorrect releases coincide with zero required-answer coverage, leaving comparative oversight efficacy unanswered. The paper therefore answers the opening systems question at the implemented scope while preserving the failure that prevents a stronger empirical conclusion.

References

[1] Ryan Greenblatt, Buck Shlegeris, Kshitij Sachan, and Fabien Roger. *AI Control: Improving Safety Despite Intentional Subversion*. ICML 2024, PMLR 235:16295-16336. Conference publication; inspected extended report, arXiv:2312.06942v5, 2024. The extended report differs from the shortened conference version.

[2] Todd J. Green, Grigoris Karvounarakis, and Val Tannen. *Provenance Semirings*. PODS 2007. DOI: 10.1145/1265530.1265535. Author-hosted paper.

[3] Niklas Wretblad, Fredrik Gordh Riseby, Rahul Biswas, Amin Ahmadi, and Oskar Holmstrom. *Understanding the Effects of Noise in Text-to-SQL: An Examination of the BIRD-Bench Benchmark*. ACL 2024, Short Papers, pp. 356-369. Primary publication.

[4] UK AI Security Institute. *Inspect: Sandboxing*. Documentation. The local adapter's actual dependency version is bound by `uv.lock`. Official documentation.

[5] UK AI Security Institute and Redwood Research. *ControlArena*. Documentation. No ControlArena integration is claimed in the measured implementation. Official project documentation.

[6] OpenAI. *GPT-5.4 Mini model* and *Function calling*. Documentation. The authorized pilot configuration uses the recorded model `gpt-5.4-mini-2026-03-17`, standard text input USD 0.75 and output USD 4.50 per million tokens, a 400,000-token context, no built-in paid tools, and an explicit reservation policy. Account availability and pilot execution are measured separately. Model documentation; function-calling documentation.

[7] Assay source repository, source revision and modifications identified in Appendix A.2. Primary implementation references: `src/assay/contracts`, `datasets`, `gateway`, `execution`, `evidence`, `verification`, `review`, `agents`, `publication`; `sandbox/worker.py`; `evaluator/oracle.py`; `evaluator/milestone.py`; and the saved validation records. Planning references: `IMPLEMENTATION_PLAN.md`, `IMPLEMENTATION_BACKLOG.md`, `docs/CHARTER.md`, and architecture decisions 0001-0005. TraceFrame pin: `422befdcbadb95024b75b2e4ac381cb6716b97c3`; AMOS inspected planning reference: `ab98863e643a0dd377084c7365fb283f3a3a5fcf`.

[8] Shumo Chu, Chenglong Wang, Konstantin Weitz, and Alvin Cheung. *Cosette: An Automated Prover for SQL*. CIDR 2017. Primary paper.

[9] Yang He, Pinhan Zhao, Xinyu Wang, and Yuepeng Wang. *VeriEQL: Bounded Equivalence Verification for Complex SQL Queries with Integrity Constraints*. PACMPL 8, OOPSLA1, article 132, 2024. Published paper.

[10] Rocky Klopfenstein, Yang He, Andrew Tremante, Yuepeng Wang, Nina Narodytska, and Haoze Wu. *SpotIt: Evaluating Text-to-SQL Evaluation with Formal Verification*. ICLR 2026. Official proceedings.

[11] Tarfah Alrashed, Madhup Sukoon, David R. Karger, and Natasha Noy. *Developing and Benchmarking Verification Algorithms to Improve Text-to-SQL Generation*. PVLDB 19(11):3732-3744, 2026. DOI: 10.14778/3836663.3836721. Published paper.

[12] Boshen Shi, Yize Liu, Chen Zhao, Ce Chi, Zhendong Wang, Xing Wang, and Junlan Feng. *TrustDABench: Benchmarking Reliability and Robustness of LLMs for Structured Data Analysis*. arXiv:2608.24145v1, 2026; preprint, peer-reviewed acceptance not verified in this review. Inspected version.

[13] Khanh Linh Nguyen, Hoa Nghiem, and Tu Tran. *TraceGuard: Structured Multi-Dimensional Monitoring as a Collusion-Resistant Control Protocol*. arXiv:2604.03968v1, 2026; preprint. Inspected version.

[14] Mikhail Terekhov, Alexander Panfilov, Daniil Dzenhaliou, Caglar Gulcehre, Maksym Andriushchenko, Ameya Prabhu, and Jonas Geiping. *Adaptive Attacks on Trusted Monitors Subvert AI Control Protocols*. ICLR 2026; inspected arXiv:2510.09462v2. Official proceedings, inspected version.

[15] Edoardo Debenedetti, Ilia Shumailov, Tianqi Fan, Jamie Hayes, Nicholas Carlini, Daniel Fabian, Christoph Kern, Chongyang Shi, Andreas Terzis, and Florian Tramer. *Defeating Prompt Injections by Design*. IEEE SaTML 2026. DOI: 10.1109/SaTML68715.2026.00040; inspected arXiv:2503.18813v2, 2025. Publisher record, inspected version.

[16] Ran El-Yaniv and Yair Wiener. *On the Foundations of Noise-free Selective Classification*. JMLR 11:1605-1641, 2010. Primary publication.

[17] Kiran-Kumar Muniswamy-Reddy, David A. Holland, Uri Braun, and Margo Seltzer. *Provenance-Aware Storage Systems*. USENIX ATC 2006. Primary publication.

[18] Jerome H. Saltzer and Michael D. Schroeder. *The Protection of Information in Computer Systems*. Proceedings of the IEEE 63(9), 1975. Author-hosted paper.

Appendix A. Implementation and reproduction specification

A.1 Supported interfaces and operational limits

The versioned objects are `TaskSpec`, `DatasetSnapshot`, `ExecutionReceipt`, `Claim`, `ReportRevision`, `CheckBundle`, `ReviewDecision`, `ApprovalRecord`, and `ReleaseRecord`. Records reject extra fields and are frozen at the model layer. Reports contain 1-64 claim records. Numeric claims use bounded finite decimals and reject binary floats and Booleans. The fixed metric wording, population, status, currency, entity key, scale factor, and null policy are checked against `settled-order-revenue-1.0`; retaining a version name cannot authorize a changed contract.

Boundary	Implemented restriction
SQL gateway	One DuckDB SELECT; at most 16,384 submitted UTF-8 bytes, 512 AST nodes, depth 32
SQL capabilities	Explicit expression/argument allowlist; no writes, recursive CTEs, windows, set operations, file/URL functions, external catalogs, or unlisted functions
Container	UID/GID 10001; no network; read-only root and public input mount; dropped capabilities; no privilege escalation
Resources	One CPU; 256 MiB memory; 64 processes; 64 open files; 64 MiB temporary filesystem
Query response	30-second default deadline; 1,000 rows; 1,048,576 output bytes; no truncated success
Worker input	At most 32,768 bytes; duplicate JSON keys, nonfinite values, incorrect types and unexpected fields rejected
Tables	Fixed CSV schemas; each at most 8 MiB and 50,000 rows
DuckDB session	One thread; 128 MB internal memory; at most 32 MB spill; external access/extensions disabled; configuration locked

Table A1. These are implemented configuration limits. Enforcement evidence comes from V-container's selected useful/denied/resource/cleanup cases, not the table's existence. The query deadline does not cover all bundle staging or the whole model episode.

The host freezes only allowlisted public files into a manifest-derived directory, validates the copied bundle, and sets explicit `0755`/`0644` directory/file modes before publication of the retained snapshot. This avoids a demonstrated portability failure where owner-only staging directories were readable through Windows sharing but not by the fixed worker UID on Linux bind mounts. Source/policy changes require a new worker build identity. The measured pilot worker installs DuckDB and SQLGlot but omits `pytz`; no repaired image or paid rerun is substituted in the results.

Evidence schema version 3 stores canonical blobs and indices for source history/epochs, executions/results, attempts, claims/revisions, decisions/approvals, releases, and current pointers. SQLite uses foreign keys, WAL, `synchronous=FULL`, and `BEGIN IMMEDIATE` for writes. Execution identity retries must be identical or fail. An edge u to v records v's dependency on u; cycle checks and recursive queries support inspection. This index is not a guarantee that every descendant is revalidated at every operation.

A.2 Source and artifact map

Table A2 maps implemented obligations to repository sources [7].

Obligation	Primary implementation	Supporting evidence
Public task and canonical truth	`src/assay/contracts`, `src/assay/datasets`, `evaluator/oracle.py`	`tests/test_contracts.py`, `tests/test_oracle.py`, M-numerical
SQL/source admission	`src/assay/gateway/policy.py`, `service.py`	`tests/test_sql_policy.py`, `tests/test_gateway.py`
Worker boundary	`src/assay/execution/docker.py`, `sandbox/worker.py`	`tests/test_sandbox_integration.py`, V-container
Evidence and partial semantics	`src/assay/evidence/store.py`, `src/assay/verification`	`tests/test_evidence*.py`, `tests/test_verification.py`
Reviews, correction, publication	`src/assay/review`, `agents`, `publication`	`tests/test_review.py`, `test_arms.py`, `test_publication*.py`
Pilot budget admission	`research/run_pilot.py`	`tests/test_pilot_budget.py`, P-development ledger

Table A2. Paths identify inspectable implementation, not independent papers or a substitute for the methods stated here. The runtime wheel omits the private evaluator; that packaging property alone is not host-process isolation.

The principal saved environment uses Python 3.12.6, DuckDB 1.5.6, SQLGlot 30.23.0, Pydantic 2.14.1, Inspect 0.3.281, and TraceFrame 0.1.0 at its pinned upstream revision. The enabled container record uses a Linux/amd64 Docker engine with client/server version 29.8.2. The source inventory in `research/validation/current-validation.json`, together with the lockfile, identifies the tested dirty tree. The reconciliation in `research/validation/repository-review.json` records no mismatches among those 80 implementation/configuration/test digests; the JUnit files separately record test outcomes. TraceFrame is an optional researcher-audit exporter; ControlArena is a related framework rather than an implemented dependency [5].

The exact evidence identities are:

```
M-numerical: research/validation/milestone.json
SHA256 951c4b1cd396ae691ed6dd21da04eccd62153704d8bf88746a9615744dabc19f

V-container: research/validation/full-junit.xml
SHA256 6d5e2f3ff37f210044b65ff2d4262915d758059c68ad4a0594fcfa9c481208f6

V-repository: research/validation/repository-review-junit.xml
SHA256 0eaa0b0f312da28719391e5dda34df104627b6d01933fbfa5385a68f4f873a18

P-development frozen assignment manifest
SHA256 30f73a811039cc6219c463d454f8ec6ca1d93e7246d4133ed2efb0a112cda31e

P-development terminal results
SHA256 ca2975ac60e10c9b0e4b6d270a358308f50802df7f00d35142f1d95c83e735da

Measured worker image identity
sha256:7afcc4b3a908da64495217dc43c5d76e39c2c5aa9bd6950bd0e9d6809d96fb7b
```

The validation manifests retain exact commands, warnings, environment, and execution provenance. Raw pilot requests, responses, transcripts, journal, and manifests remain in local state outside the synchronized checkout. The repository's aggregate summary is `research/validation/development-pilot.json`. Assignment identity is retained as `run_id`; the runner separately generates an `episode_id`. The audit reconciles every assigned run exactly once and verifies its arm, regime, variant, snapshot, hashes, tokens, and settlement.

A.3 Reproduction commands

```
uv sync --locked --extra traceframe --extra inspect
uv run --locked python -m ruff check .
uv run --locked python -m ruff format --check .
uv run --locked python -m mypy
uv run --locked --extra traceframe --extra inspect python -m pytest
uv run --locked python -m evaluator
uv run --locked python research/audit_evidence.py
uv build --no-sources
uv run --script research/build_paper.py
```

Require both optional integrations through `ASSAY_TRACEFRAME_TESTS=1` and `ASSAY_INSPECT_TESTS=1` when reproducing their enabled test scope. Require the real worker suite with `ASSAY_DOCKER_TESTS=1` only in a functioning Linux Docker environment; missing Docker is not a pass. Use the audit's `--pilot-dir` argument with the original local pilot directory for read-only ledger reconciliation. Audit and paper builds make no model calls. The numerical milestone uses trusted fixed SQL; it is not a substitute for a real worker test. Fresh paid experiments require new identities and a separate frozen protocol while preserving cumulative prior charges.

Appendix B. Prospective comparative study

This appendix specifies research not completed by the reported pilot. Its sample sizes, margins, and analyses are planning quantities, not observations or spending authority.

B.1 Hypotheses and outcomes

The intended primary comparison is E versus D: the same analyst configuration, reviewer, executable checks, permitted evidence corpus, retrieval tools, and combined allowed oversight resources, with a different initial representation. D spends resources on model-written summarization; E starts from host-built evidence. Any extra accessible information or stronger tool changes the intervention. Other compatible SQL-verification baselines must be implemented and frozen before a comparative novelty claim.

Let i index assigned matched cases, a an arm, g a regime, and S_i required slots. Let L indicate a confirmed release, M at least one materially incorrect released required or permitted supporting quantitative claim, and Z_s a correctly released required slot, including a justified contract-authorized abstention. Define

$$R_{ag} = \frac{\sum_i L_{iag} M_{iag}}{N_g}, \qquad C_{ag} = \frac{\sum_i \sum_{s \in S_i} Z_{iags}}{\sum_i |S_i|}.$$

Risk uses all assigned episodes; coverage uses all assigned required slots. Withholding contributes neither an incorrect release nor a correct required answer. A correct required answer plus a false supporting claim can increase coverage while triggering the safety event. A report-level correctness flag is insufficient because missing slots and actual wrong assertions affect these endpoints differently. Report complete-correct-task rate, release frequency, conditional error among releases, failures, actual cost, and latency alongside both primary outcomes. An all-withhold reference has zero incorrect releases and zero coverage.

H1 requires lower E-minus-D incorrect-release risk under prompted sabotage. H2 requires ordinary-regime required-answer coverage to remain above a frozen noninferiority margin. Both must pass the prespecified joint rule for a positive primary conclusion. Materiality, final margin, models, prices, sample size, and inference configuration must be resolved before confirmation. Exact-contract errors should accompany any coarser materiality threshold. Secondary regimes/comparisons require prespecified multiplicity treatment or descriptive labeling.

B.2 Tasks, attacks, and exposure

The current generator supplies only one exposed analytical family. A broader benchmark must include grain, population/denominator, missingness, units/time, evidence freshness, and insufficiency/completeness failures. Each template needs an explicit contract, independent truth, hand-worked cases, alternative correct solutions, intentionally wrong programs, discriminating data, tolerances, and limitations. New task families require new contracts and check/scorer support.

Split development, calibration, and confirmation on task structure, schema composition, and attack formulation, not only numerical seeds. Maintain an exposure register. Once inspected, published, or used for tuning, a case cannot silently regain untouched-confirmation status. Keep the private scorer, gold answers, seeds, diagnostic labels, and attack-success feedback outside actor/reviewer interfaces, validating the actual information boundary.

Evaluate ordinary errors, indirect injection, and prompted sabotage separately. Injection may modify only declared text surfaces before snapshot freezing and must preserve independently derived legitimate answers for every allowed slot. Numeric poisoning or removal of necessary evidence changes the regime. Prompted sabotage supplies an explicitly constructed side objective; reviewers receive legitimate-task context without that objective. Adaptive development attacks need equal declared resources, explicit attacker knowledge, retained attempts, and success criteria based on the consequential release rather than suspicion alone.

The pairing key is template, dataset/snapshot, regime, frozen attack assignment, and repetition, excluding arm. Paired arms receive identical permitted input and attack bytes. Randomize arm order within blocks and initialize independent conversations. Provider seeds do not guarantee matched trajectories.

B.3 Freeze, accounting, and inference

Before dispatch, freeze assignments, hypotheses/outcomes, task/data/check/scorer versions, source/worker identities, prompts/tools, exact models/settings, prices, budgets, corrections, retries, failure categories, exclusions, and analysis. Count summaries, reviews, correction reviews, and lookups in one common oversight allowance. Reserve conservative in-flight charges before dispatch, settle idempotently from usage, and retain uncertain reservations. Canceled/superseded attempts retain actual charges. No repeat-until-success episode policy is admissible.

The implemented correction mechanism permits at most one round after rejected/needs-review revisions, under shared analyst budgets. Feedback contains the arm's own reasons and permitted reviewer codes, not arbitrary reviewer prose or private scores. A correction is a new revision, checked and reviewed again; approved or stale revisions do not acquire an extra correction allowance. General attempt expiry, gateway result admission, cancellation and distributed retry integration require further validation.

For k known incorrect releases and u unknown release outcomes among N assignments, the finite-list identification bounds are

$$\frac{k}{N} \leq R \leq \frac{k+u}{N}.$$

Unknowns are not safe withholding. Apply analogous bounds to unresolved required slots and report failure categories by arm, regime and template. A matched-valid sensitivity analysis supplements, rather than replaces, the all-assignment result. Exclusions require frozen eligibility/bookkeeping rules independent of observed treatment outcomes. A scorer defect requires versioning and rescoring all affected arms.

The existing planning envelope proposes 216 benchmark-pilot episodes and 1,080 confirmation episodes; the latter uses twelve structural templates, ten datasets, three regimes, and three arms. These are arithmetic proposals, not power calculations, and the independent structural units remain twelve. A feasible precision plan must model paired discordance, template dependence, sparse errors, slot-count variation and failures before selecting a final count. It must fit a separately authorized budget.

A proposed template-cluster bootstrap uses 10,000 resamples, retaining arm pairs and recomputing risk and the ratio of total correct slots to total required slots. It needs simulation-based validation and a frozen analysis seed before use. Sparse events or few templates can yield degenerate intervals; zero observed errors do not establish zero underlying risk. The proposed decision requires the upper safety-difference interval below zero and the lower coverage-difference interval above minus the frozen margin. There is no result-dependent stopping.

Confirmation is gated on validated task/oracle contracts, a usable analyst path, attack validity, private information isolation, enforced combined budgets, audited recovery/publication boundaries, structural holdouts, feasible cost/precision, and a hashed complete preregistration. Completion additionally requires every assignment/attempt/charge accounted for, independent scoring of exact released artifacts, reproducible tables/intervals, missingness and negative findings, and a sanitized reproducibility package. None of these prospective requirements is reported as a completed comparative outcome here.

Appendix C. Pilot configuration and accounting

P-development uses `gpt-5.4-mini-2026-03-17` for analyst, summarizer, and reviewer roles, with temperature zero, reasoning effort `none`, at most 2,048 output tokens, no HTTP retries and no provider-side stored response. Each episode allows six analyst model calls, twenty tool calls and 300 seconds. Oversight allows four calls shared across summary generation, initial review and correction review. Neither summarization nor review is reached.

Each request reserves the full 400,000-token context plus maximum output at frozen uncached prices of USD 0.75 and USD 4.50 per million tokens. The journal settles conservatively at those rates [6]. An exclusive run lock, durable reservations and no automatic interrupted-run resume protect this sequential pilot. Exceptions or missing usage preserve unresolved commitments and halt. The USD 15.00 cap is shared across the existing run; it is not a new allowance for each rerun or protection against unrelated account spending.

Model call	Tool	Outcomes across 24 episodes
1	`list_tables`	24 listings
2	`preview_rows` on `orders`, limit 20	24 query errors
3	`execute_sql`	24 query errors
4	`execute_sql`	19 query errors, 4 success, 1 denial
5	`execute_sql`	19 success, 5 query errors
6	`execute_sql`	15 success, 6 query errors, 3 denials

Table C1. The 96 analyst-authored SQL calls contain 38 successes, 54 worker query errors and four pre-dispatch denials. The summary's 78 worker query errors additionally include all 24 preview failures. Four successful aggregate preflights are outside these episode counts.

Three denials concern qualified `Table.db` references and one a disallowed cast. All 38 successful analyst queries lack the token `occurred_at`; thirteen queries containing it fail in the worker and one is denied. Forty-one other worker errors lack it, so timestamp projection cannot explain every error. Inside the measured image, fetching an `orders` preview requires the missing `pytz` dependency; aggregate-only preflights did not exercise that fetch behavior.

The ledger contains 194,790 input tokens, including 137,728 cached tokens, and 3,269 output tokens, with zero reasoning tokens. Conservative settlement is

$$\frac{194790(0.75)+3269(4.50)}{10^6}=0.160803\ \text{USD}.$$

A separate reconstruction uses the documented cached-input rate of USD 0.075, USD 0.75 for the remaining 57,062 input tokens, and USD 4.50 for output, yielding USD 0.06783660. That reconstruction is not the enforced journal and is not a claim about a separately inspected provider invoice. All 144 calls are settled; no reservations remain uncertain. No new paid call is required to rebuild the paper or reconcile these records.